《算法设计与分析》 https://leanpub.com/daa/

递归方程的求解

分治递归问题的时间复杂度分析

$$T(n) = 2T(n-1)+1$$
$$T(1) = 1$$

YouTube频道: hwdong

https://github.com/hwdong-net/DAA

递归方程的求解

$T(n) = 2T(n-1)+1$
$T(1) = 1$

- 迭代展开： 迭代展开递归方程
- 递归树表示：迭代展开的可视化表示
- 假设归纳：先假设,数学归纳法
- 高阶方程的简化：转化为一阶方程
- 主定理：特殊递归方程的解

迭代展开

$$T(n) = 2T(n-1)+1$$
$$T(1) = 1$$

YouTube频道: hwdong

迭代展开

$T(n) = 2T(n-1)+1$

$T(1) = 1$

$$
\begin{aligned}
T(n) &= 2T(n-1)+1 \\
&= 2(2T(n-2)+1\ )+1 \\
&= 2(2(2T(n-3)+1\ )+1\ )+1 \\
&= \ldots \\
&= 2^{n-1}T(1)+2^{n-2}+2^{n-3}+\ldots+2+1 \\
&= 2^{n-1}+2^{n-2}+2^{n-3}+\ldots+2+1 = 2^{n}-1
\end{aligned}
$$

数列和

- 等差数列$(a_1,a_2,\ldots,a_n)$的和为: $(a_1 + a_n)n/2$

  $1+3+5+\ldots(2n-1) = n^2$

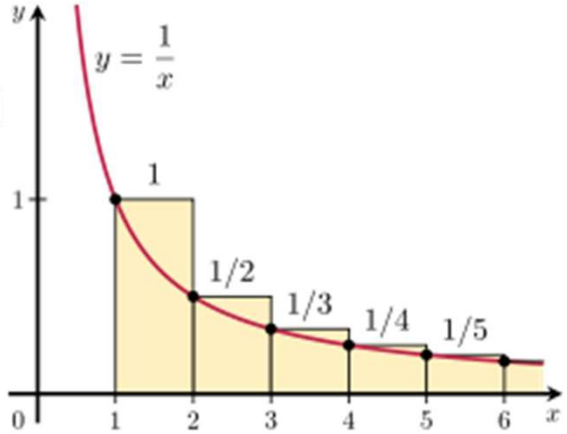
- 等比数列$(a,aq,\ldots,aq^{n-1})$的和为: $a(q^n -1) /(q-1)$

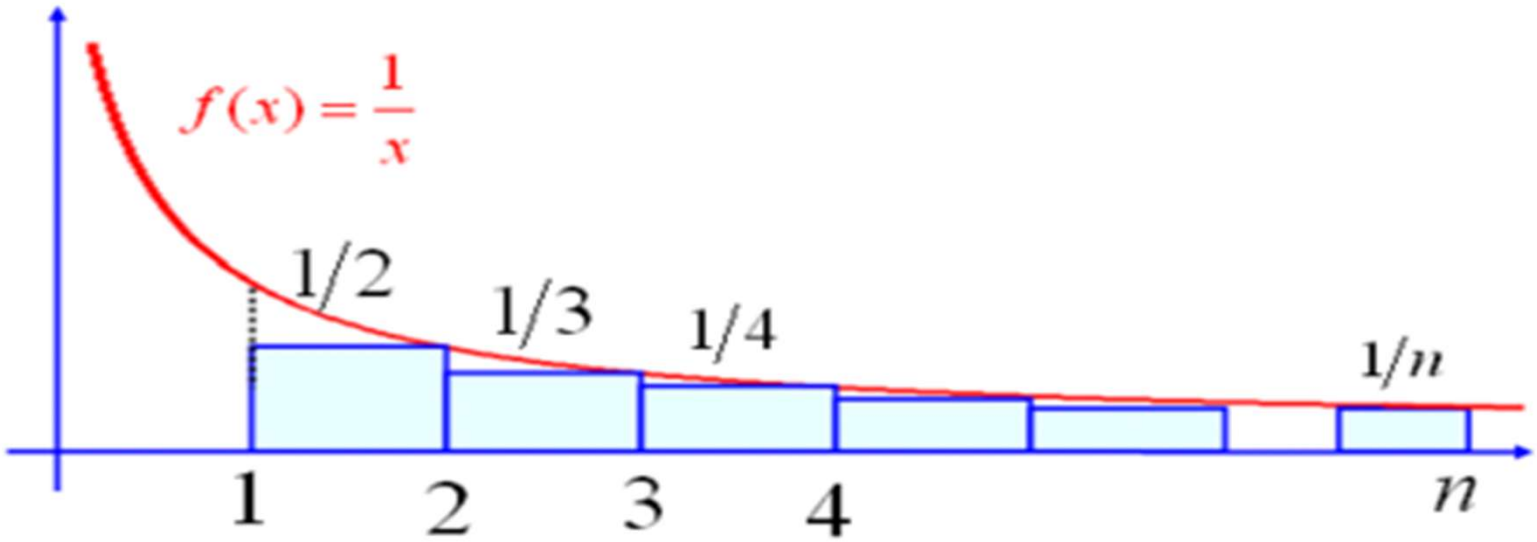
  $1+2+4+\ldots+2^n = 1*(2^{n+1}-1)/(2-1) = 2^{n+1}-1$

- 调和数列之和:

  $\ln(n+1)< 1+1/2+1/3+\ldots+1/n <1+ \ln n$

$$\sum_{n=1}^{k} \frac{1}{n} > \int_1^{k+1} \frac{1}{x}\,dx = \ln(k+1)$$





$$s_n = 1 + \frac{1}{2} + \frac{1}{3} + \ldots + \frac{1}{n} < 1 + \int_1^n \frac{1}{x}\,dx < 1 + \ln n$$

$$\therefore \quad \ln(1+n) < s_n < 1 + \ln n$$

换元迭代

$T(n) = 2T(n/2)+n-1$

$T(1)=0$

- 可令$n = 2^k$,则:

  $T(2^k) = 2T(2^{k-1})+ 2^k -1$

$$T(2^k) = 2T(2^{k-1}) + 2^k - 1$$

$$= 2(2T(2^{k-2}) + 2^{k-1} - 1) + 2^k - 1 = 2^2T(2^{k-2}) + 2^k - 2 + 2^k - 1$$

$$= 2(2(2T(2^{k-3}) + 2^{k-2} - 1) + 2^{k-1} - 1) + 2^k - 1$$

$$= 2^3T(2^{k-3}) + 2^k - 2^2 + 2^k - 2 + 2^k - 1$$

$$= \ldots$$

$$= 2^kT(2^0) + k2^k - (2^{k-1} + \ldots 2 + 1)$$

$$= nT(1) + k2^k - (2^k - 1)$$

$$= k2^k - 2^k + 1 = n\log n - n + 1$$

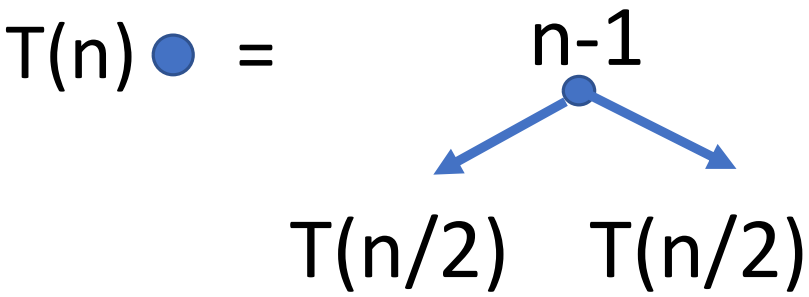
递归树表示

$$T(n) = 2T(n/2)+n-1$$
$$T(1) = 0$$

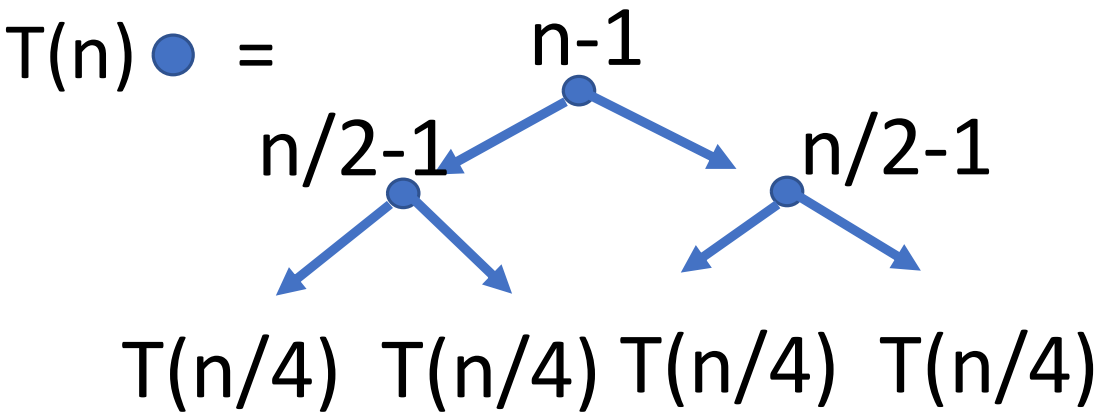
YouTube频道: hwdong

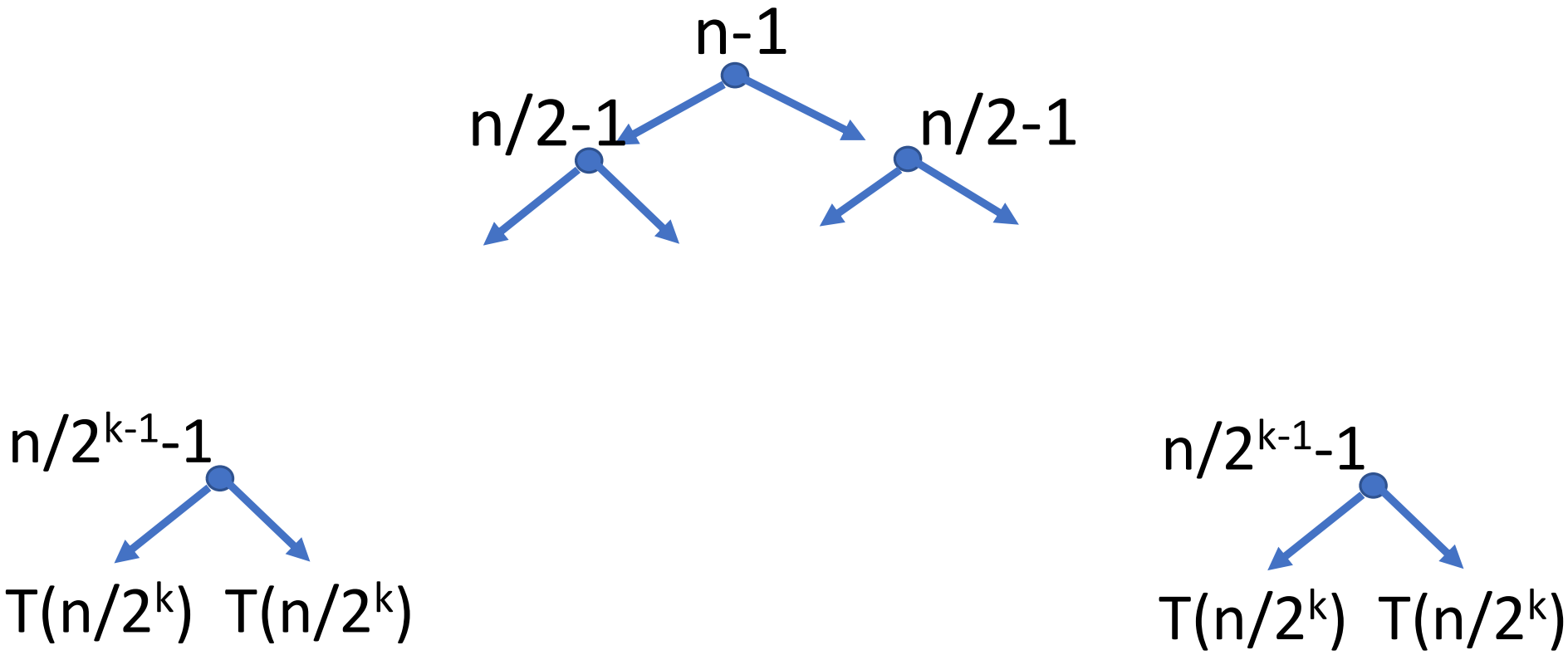
递归树表示

$T(n) = 2T(n/2)+n-1$

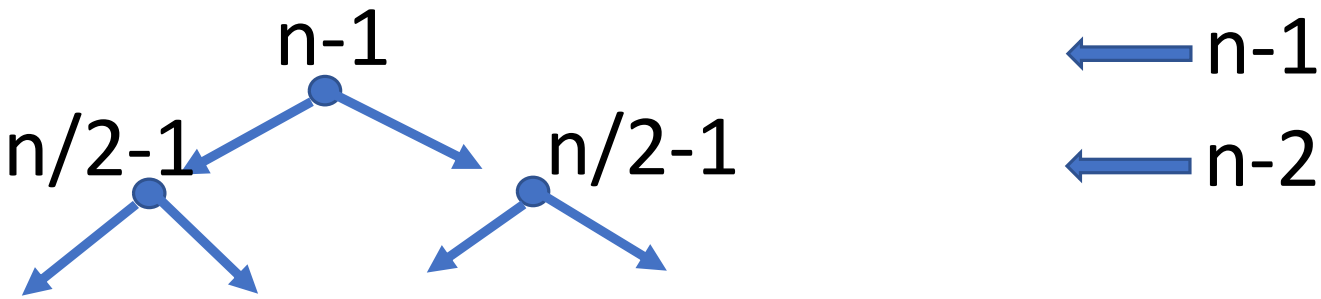


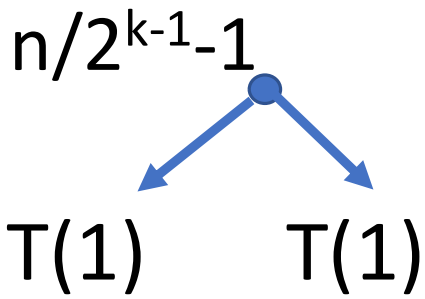
$T(n) = 2T(n/2)+n-1$

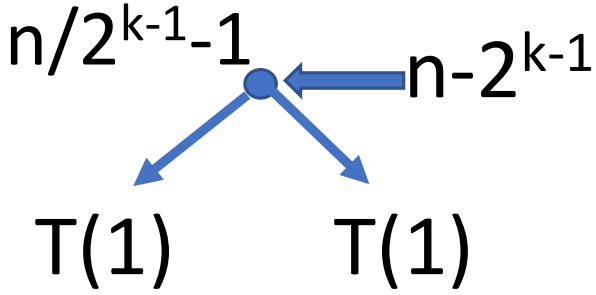



n-1
n/2-1
n/2-1
n/2^{k-1}-1
T(n/2^k) T(n/2^k)
n/2^{k-1}-1
T(n/2^k) T(n/2^k)

$$n-1 + n-2 + \dots + n-2^{k-2} + n-2^{k-1}$$
$$= kn-1-2-\dots - 2^{k-1} = kn-2^{k}+1 = n\log n - n + 1$$

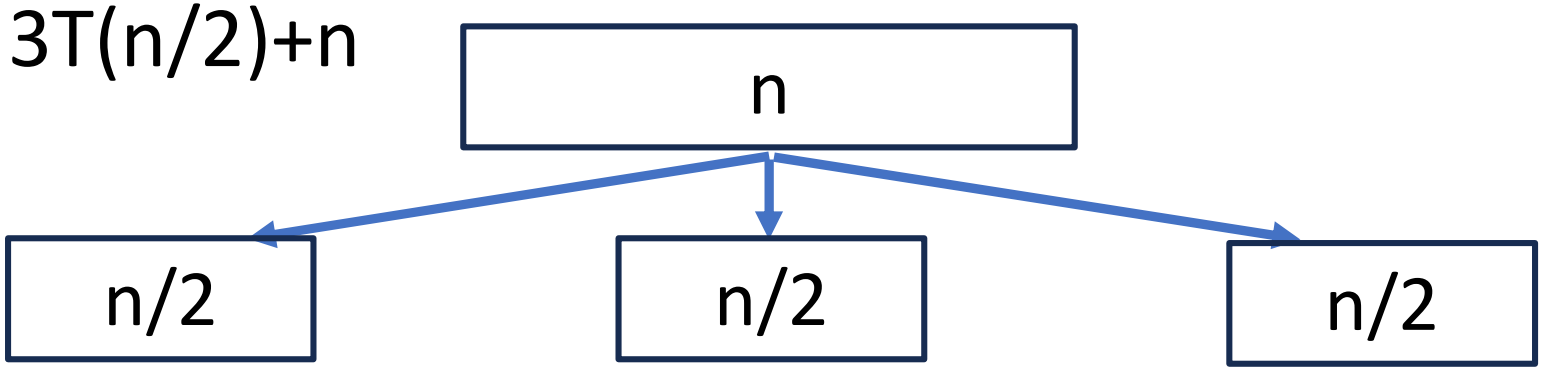




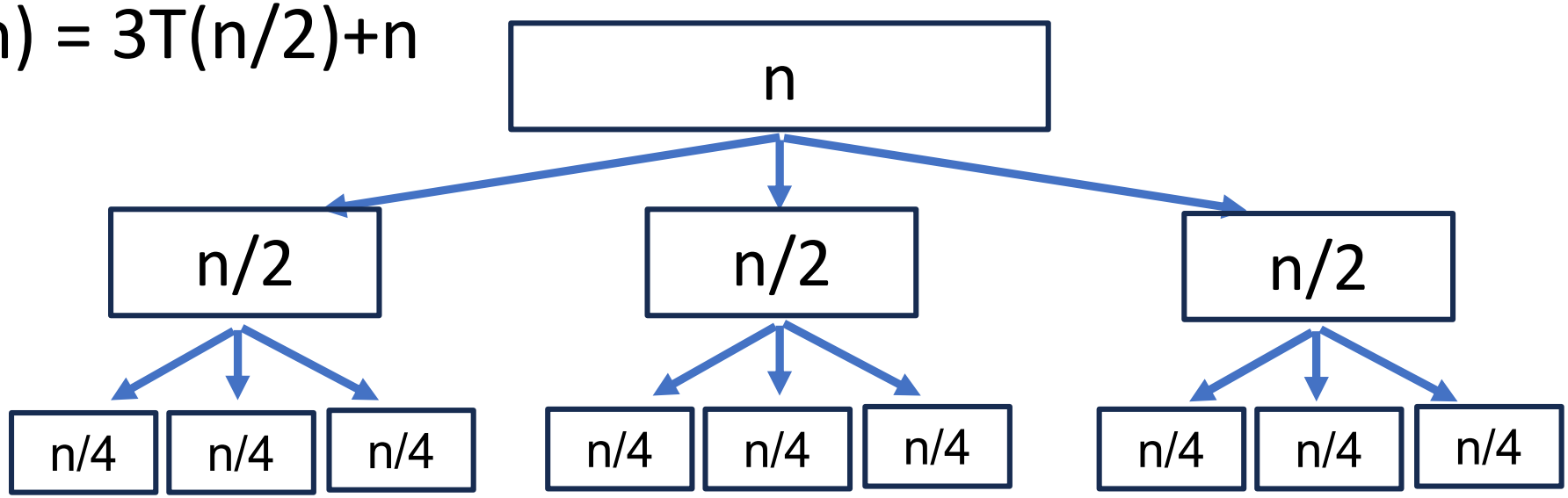


$T(n) = 3T(n/2)+n$

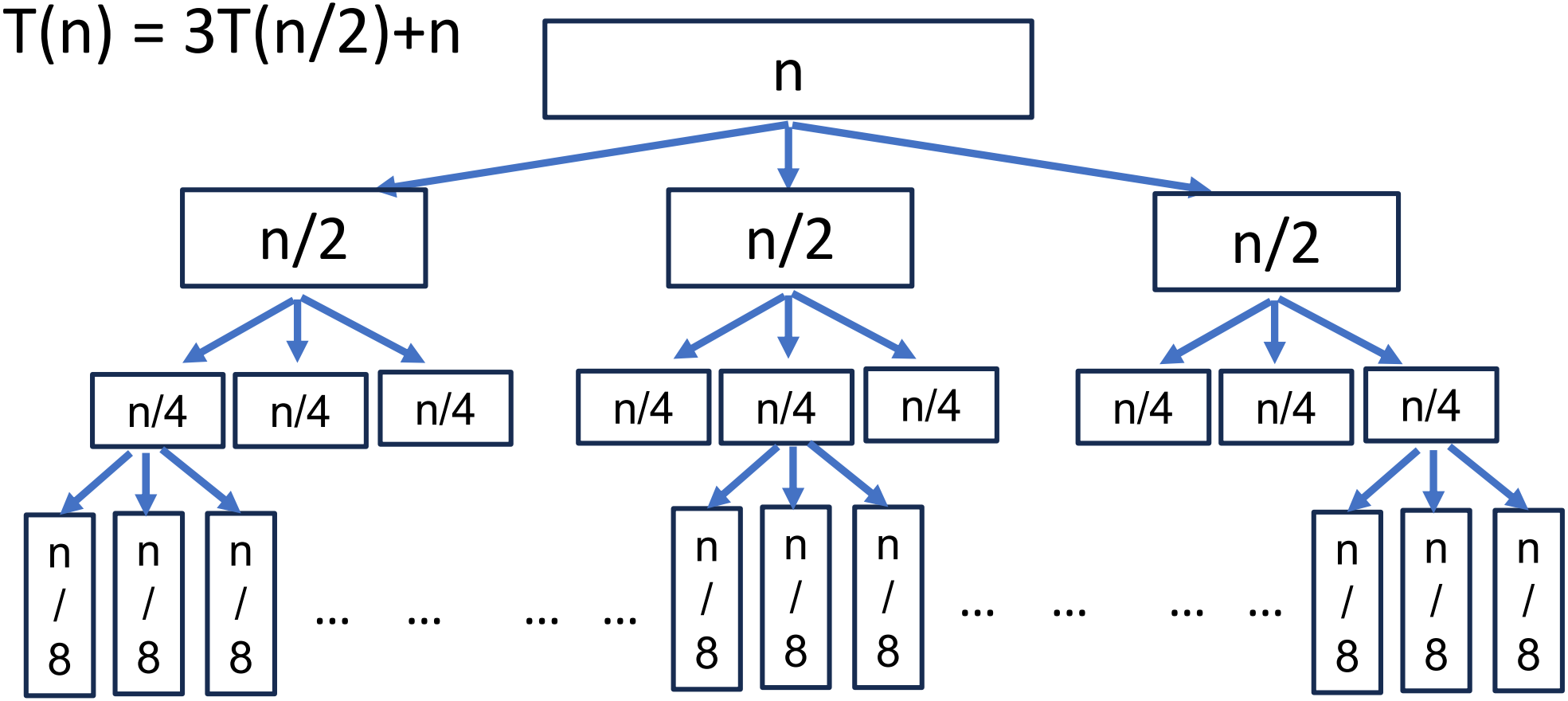
$T(n) = 3T(n/2)+n$



$T(n) = 3T(n/2)+n$

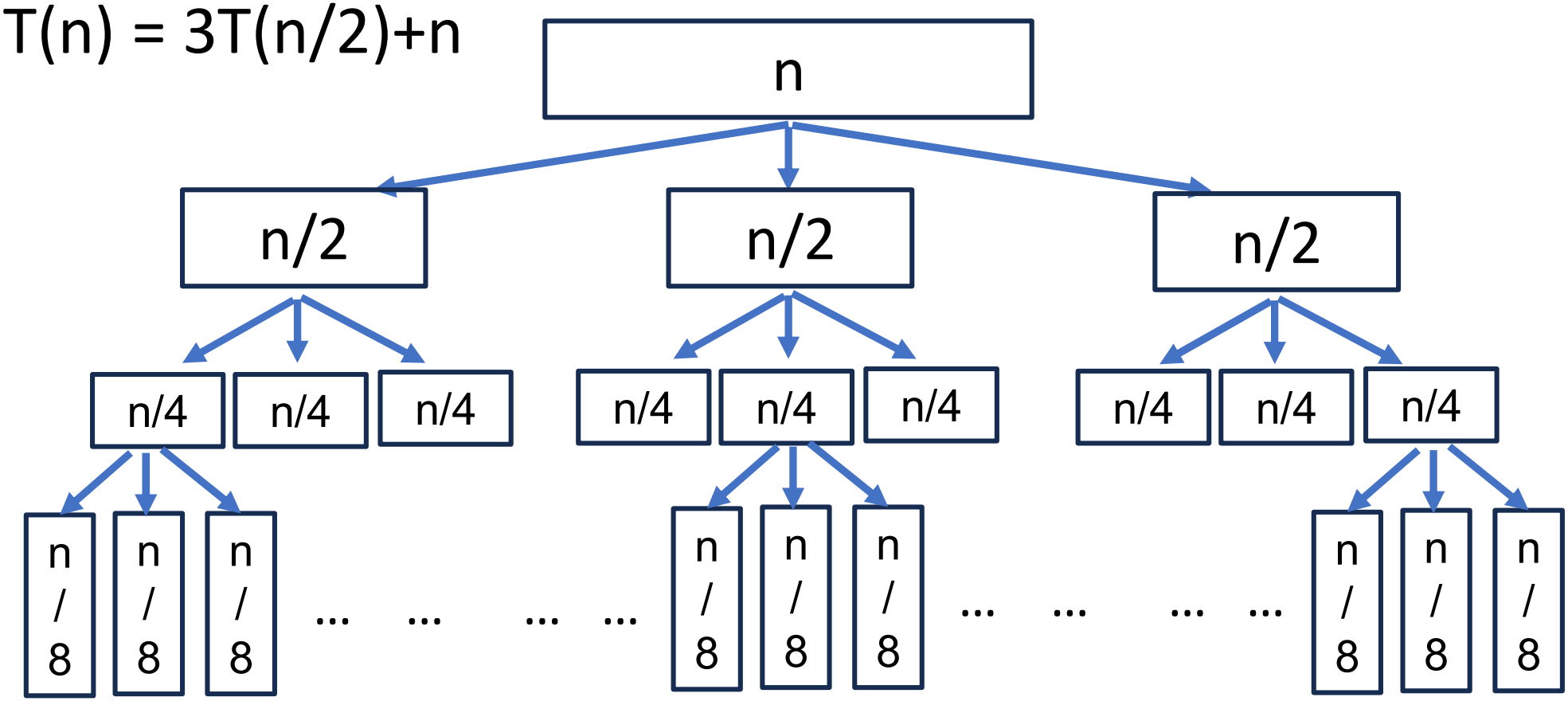


$T(n) = 3T(n/2)+n$



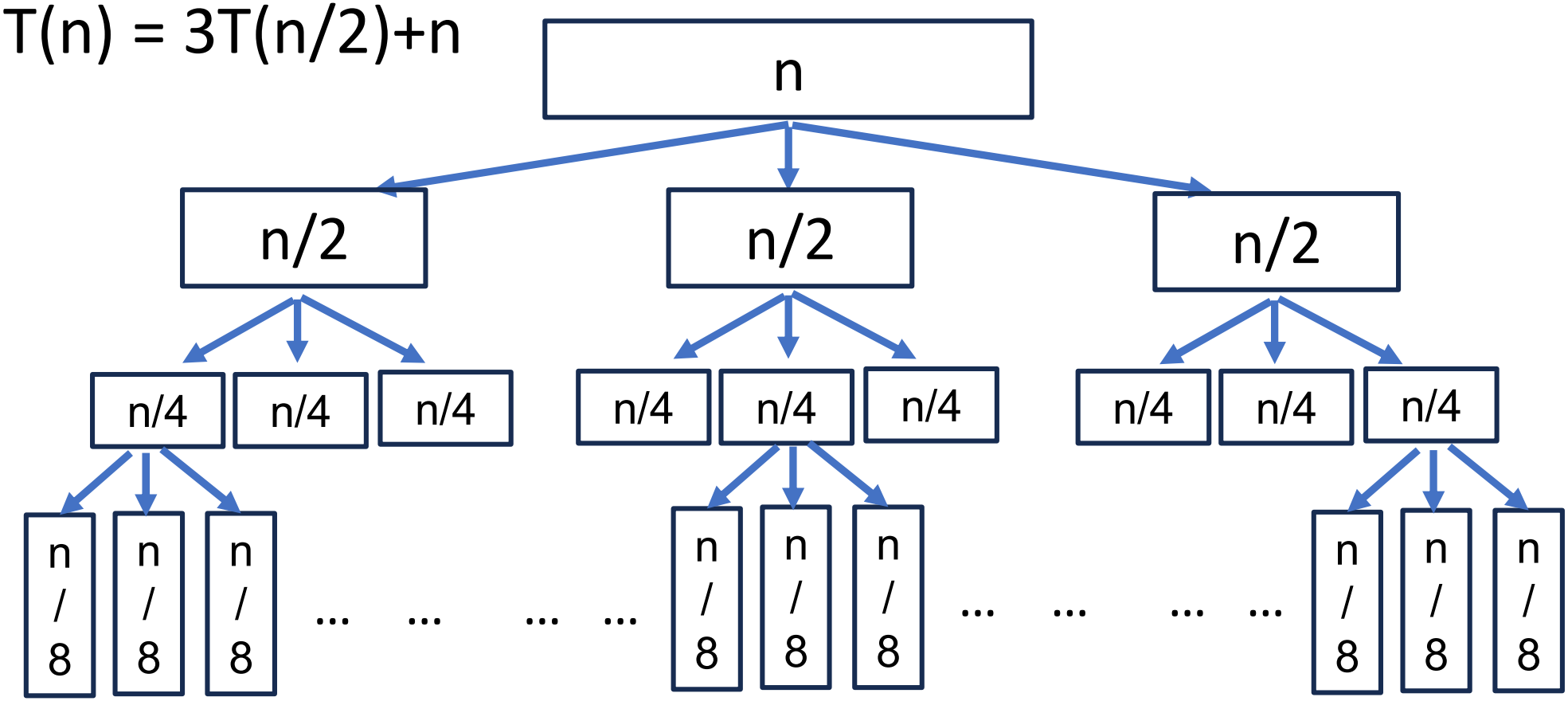
- 第i层的计算量：？

T(n) = 3T(n/2)+n



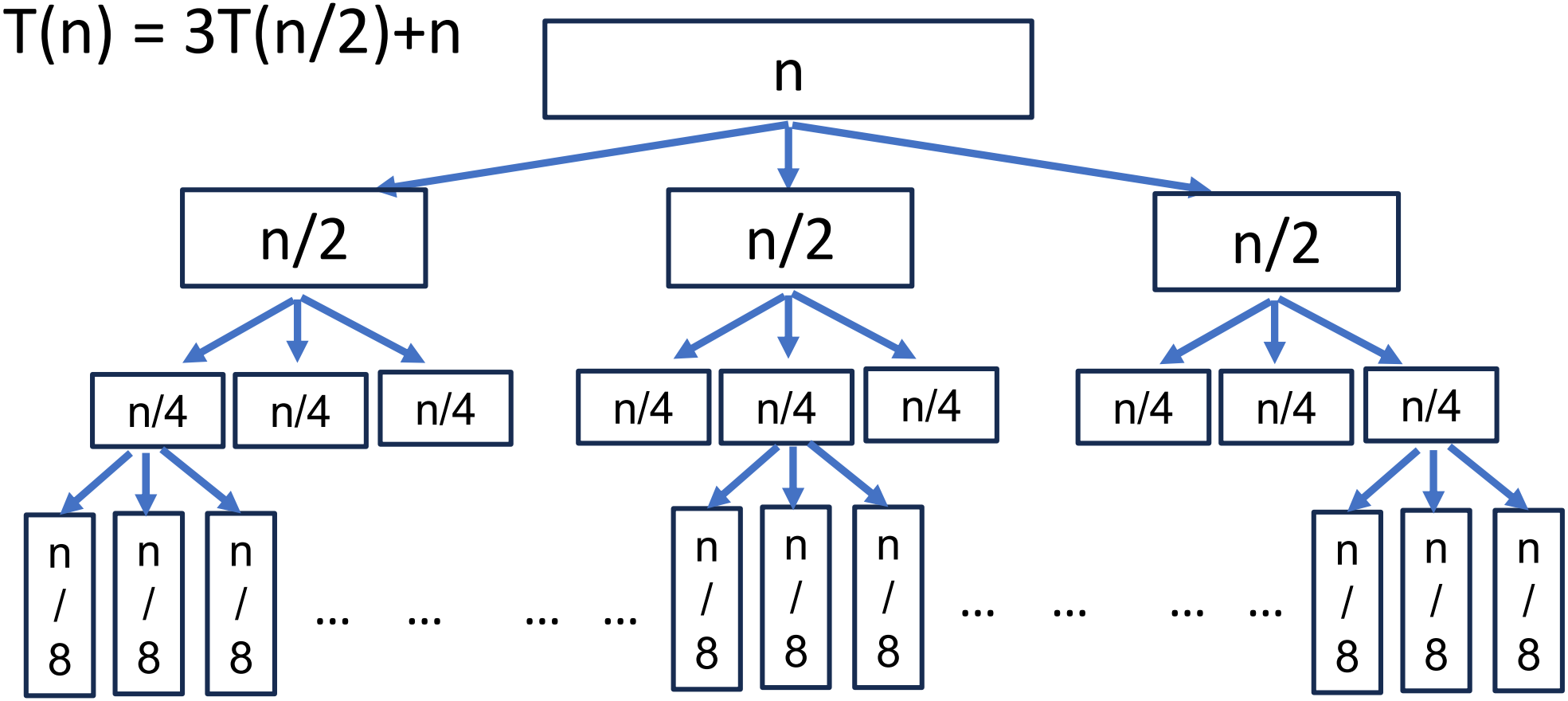
- 第i层的计算量: $\frac{n}{2^i} \times 3^i = \left(\frac{3}{2}\right)^i n$

$T(n) = 3T(n/2)+n$



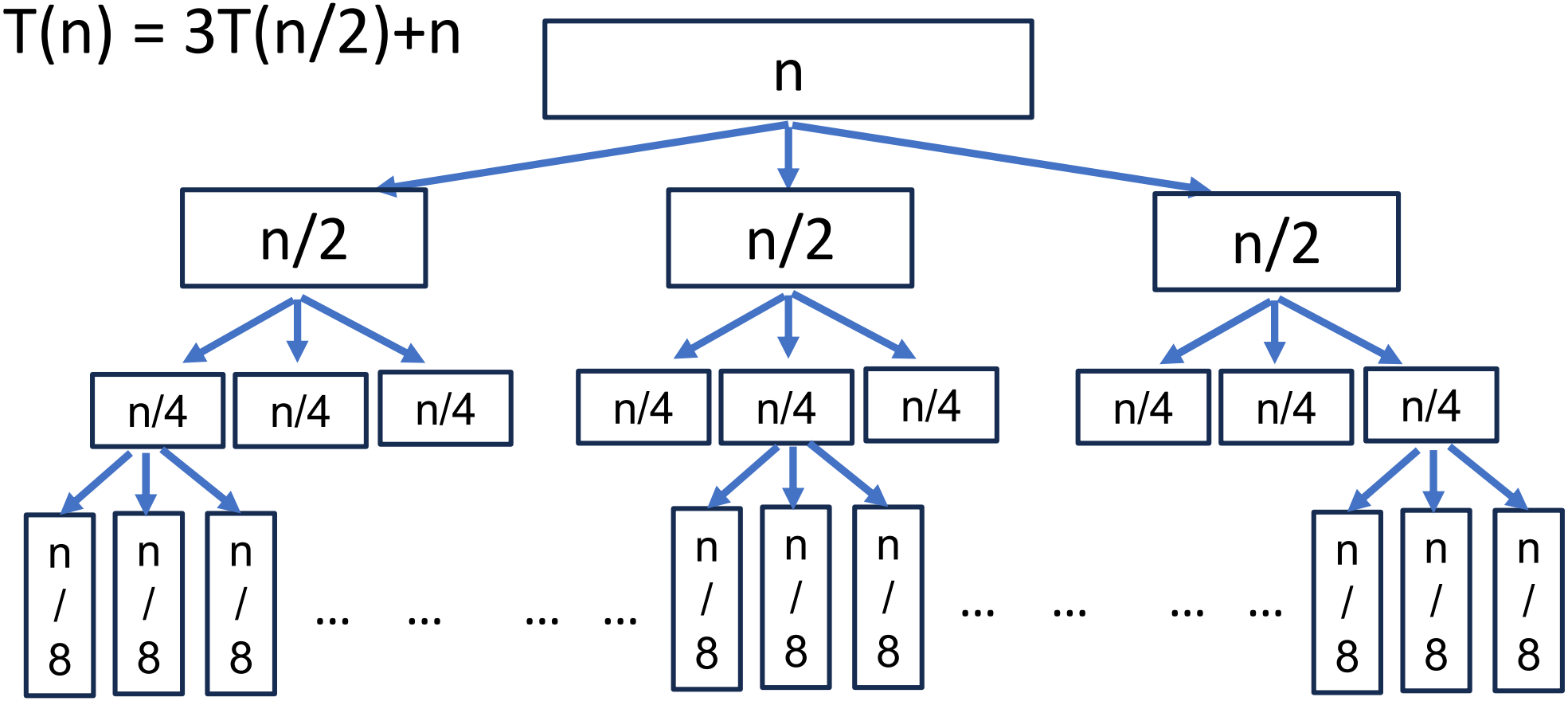
- 第i层的计算量: $\frac{n}{2^i} \times 3^i = \left(\frac{3}{2}\right)^i n$
- 最底层是第几层? $\log_2 n$

$T(n) = 3T(n/2)+n$



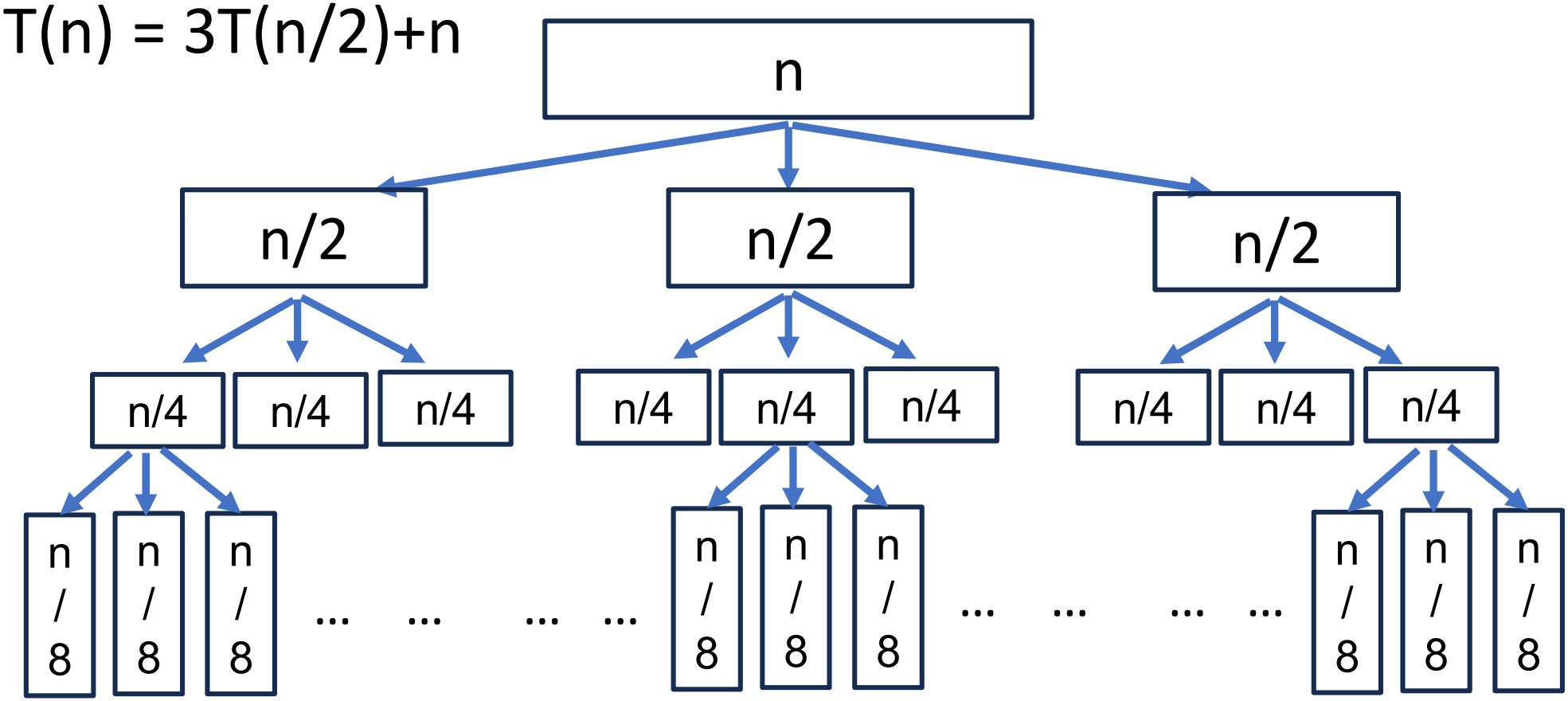
- 第i层的计算量: $\frac{n}{2^i} \times 3^i = \left(\frac{3}{2}\right)^i n$
- 最底层是第几层? $\log_2 n$
- 总计算量: $\sum_{i=0}^{\log_2 n} \left(\frac{3}{2}\right)^i n$

T(n) = 3T(n/2)+n



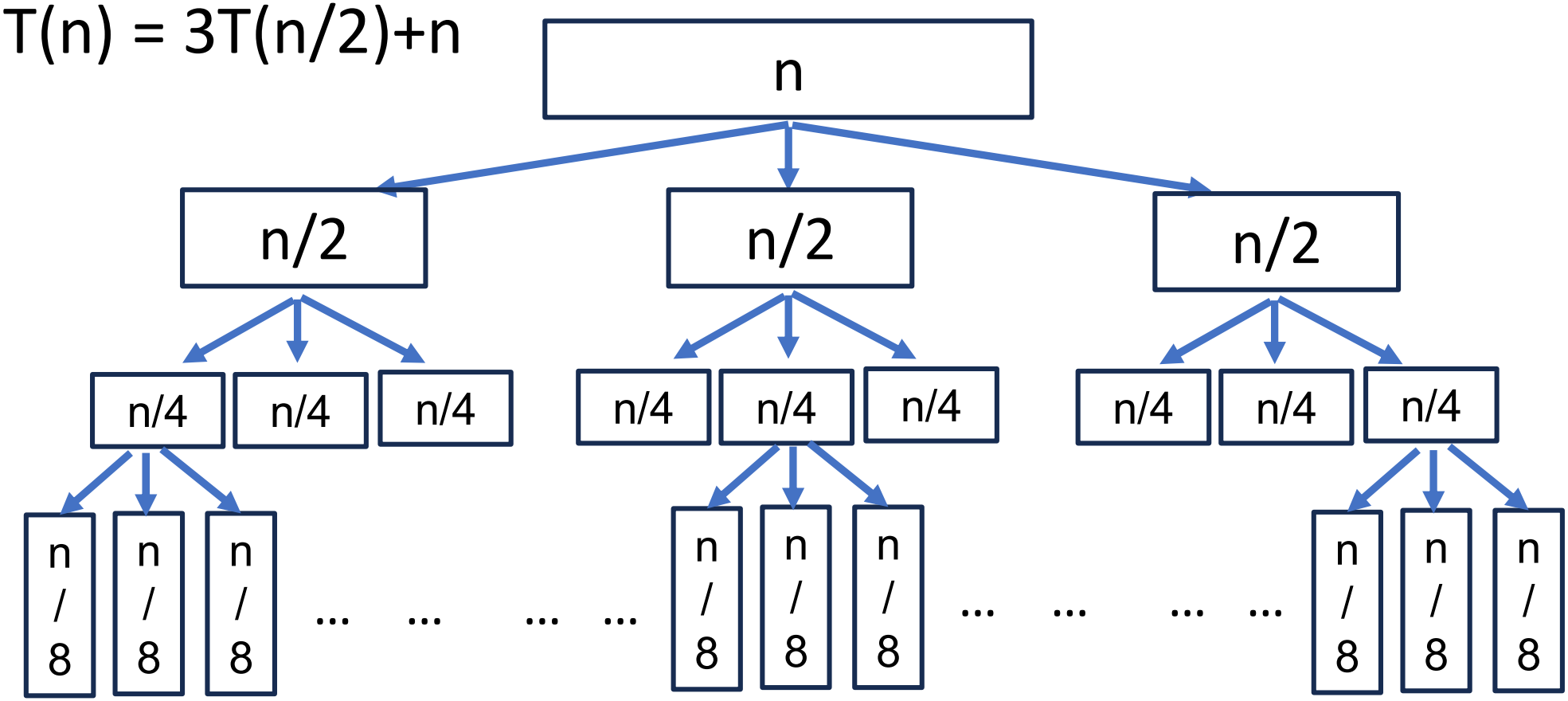
- 第i层的计算量: $\frac{n}{2^i} \times 3^i = \left(\frac{3}{2}\right)^i n$
- 最底层是第几层? $\log_2 n$
- 总计算量: $\sum_{i=0}^{\log_2 n} \left(\frac{3}{2}\right)^i n = O\left(n\left(\frac{3}{2}\right)^{\log_2 n}\right)$

T(n) = 3T(n/2)+n



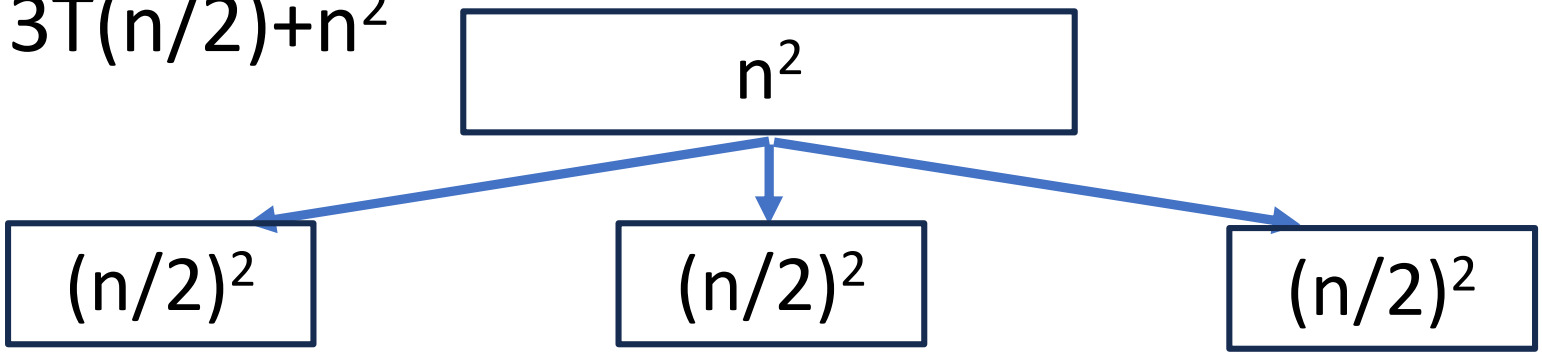
- 第i层的计算量: $\frac{n}{2^i} \times 3^i = \left(\frac{3}{2}\right)^i n$
- 最底层是第几层? $\log_2 n$
- 总计算量: $\sum_{i=0}^{\log_2 n} \left(\frac{3}{2}\right)^i n = O\left(n\left(\frac{3}{2}\right)^{\log_2 n}\right) = O(3^{\log_2 n})$

T(n) = 3T(n/2)+n

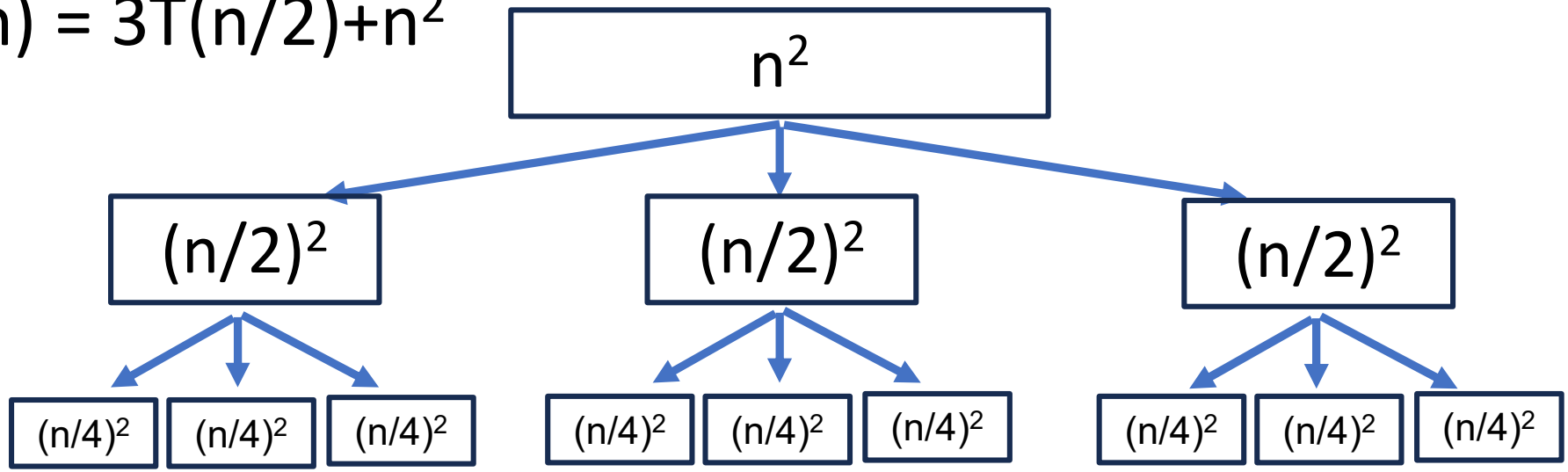


- 第i层的计算量: $\frac{n}{2^i} \times 3^i = \left(\frac{3}{2}\right)^i n$
- 最底层是第几层? $\log_2 n$
- 总计算量: $\sum_{i=0}^{\log_2 n} \left(\frac{3}{2}\right)^i n = O\left(n\left(\frac{3}{2}\right)^{\log_2 n}\right) = O(3^{\log_2 n}) = O(n^{\log_2 3})$

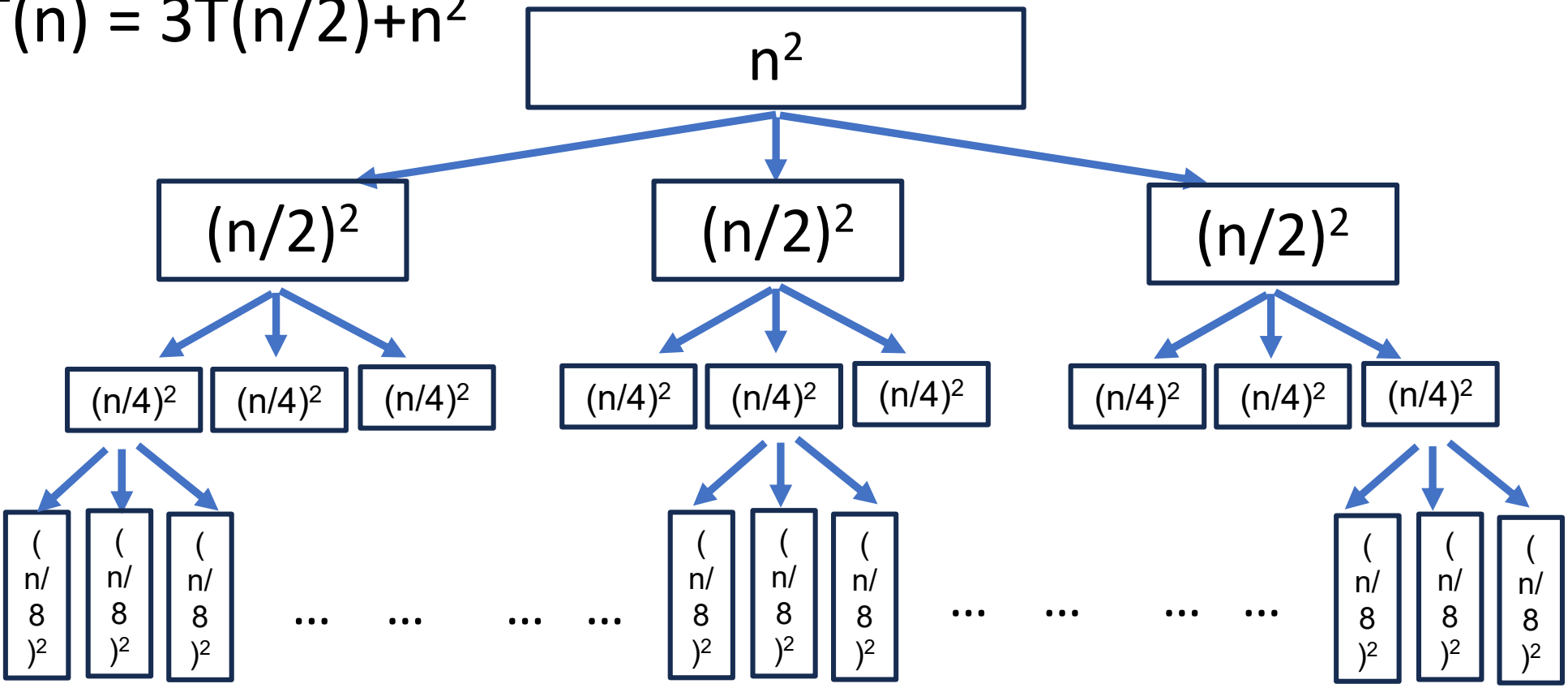
$T(n) = 3T(n/2)+n^2$



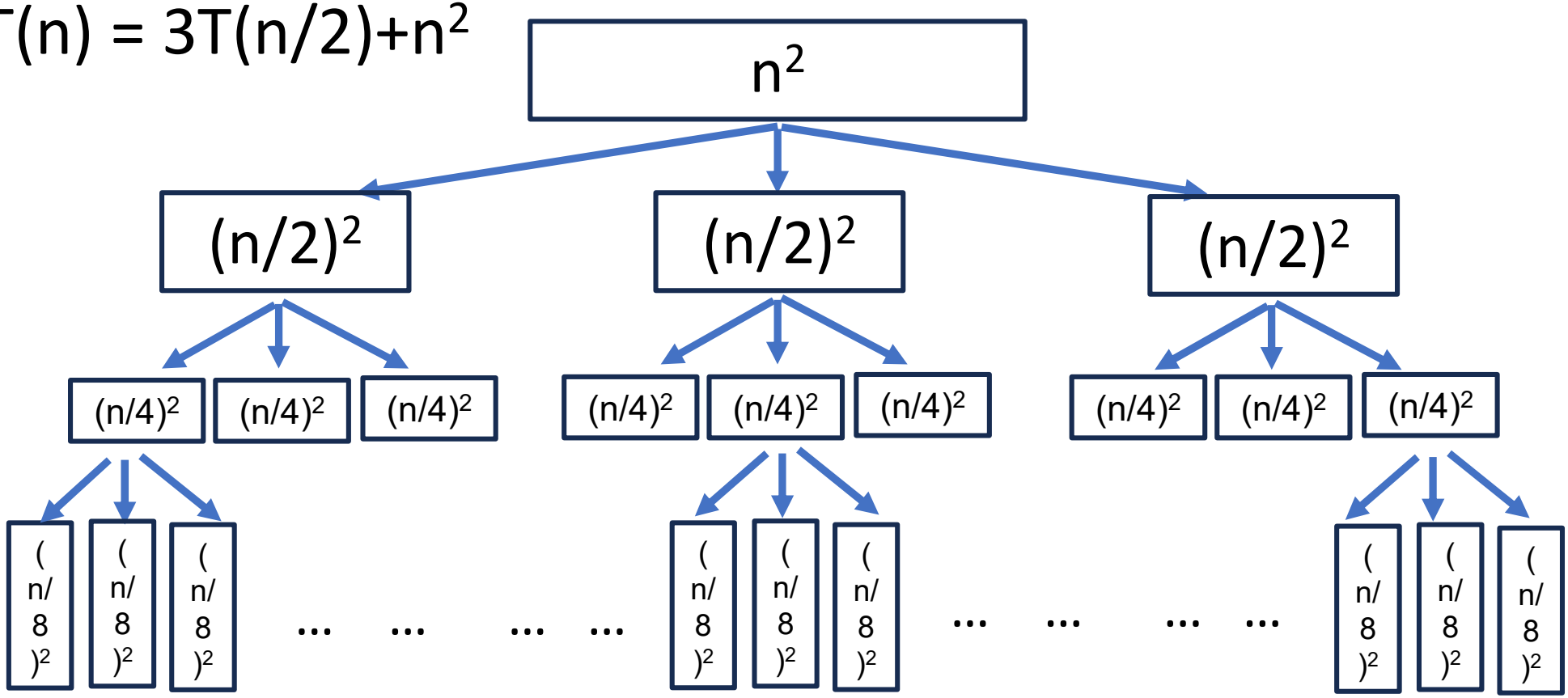
$$T(n) = 3T(n/2)+n^2$$



$$T(n) = 3T(n/2)+n^2$$

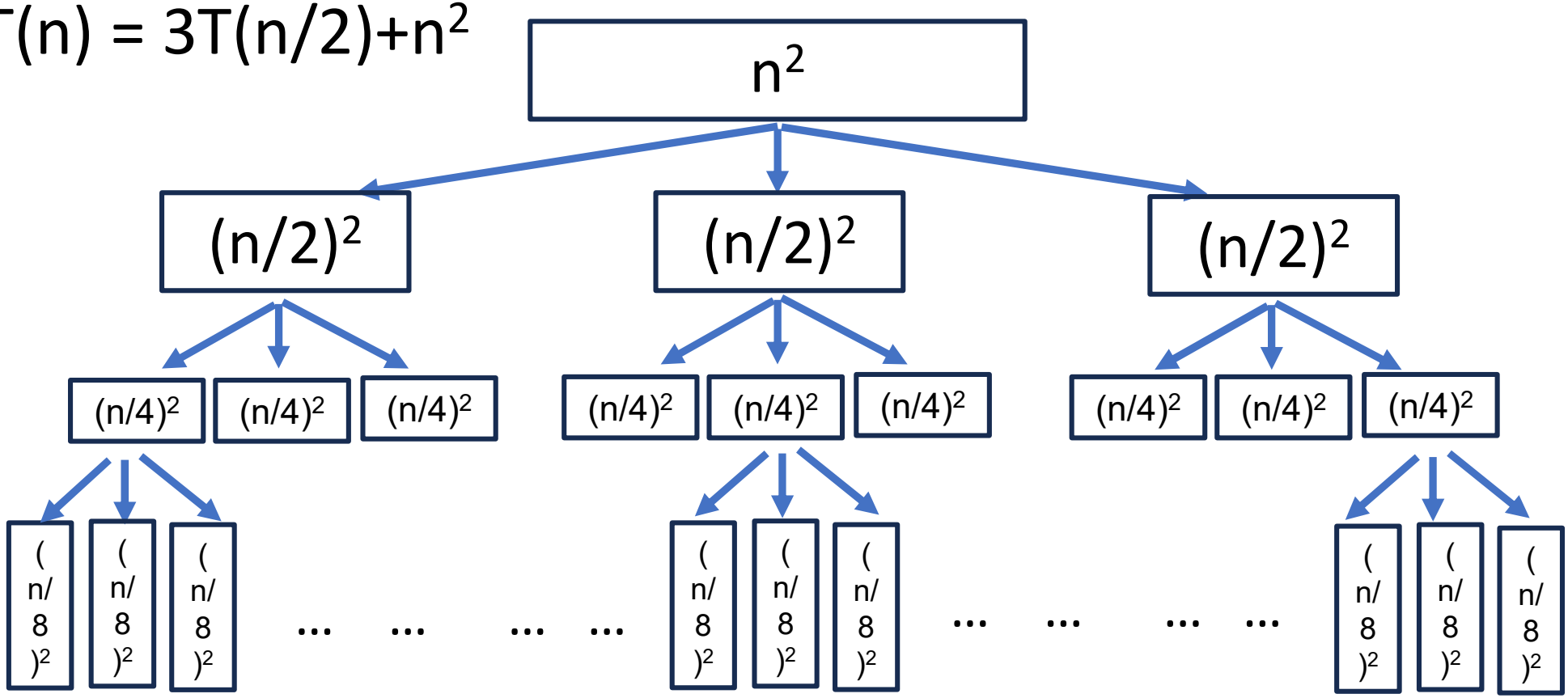

n^2
$(n/2)^2$
$(n/2)^2$
$(n/2)^2$
$(n/4)^2$
$(n/4)^2$
$(n/4)^2$
$(n/4)^2$
$(n/4)^2$
$(n/4)^2$
$(n/4)^2$
$(n/4)^2$
$(n/4)^2$
$(n/8)^2$
$(n/8)^2$
$(n/8)^2$
$(n/8)^2$
$(n/8)^2$
$(n/8)^2$
$(n/8)^2$
$(n/8)^2$
$(n/8)^2$

$T(n) = 3T(n/2)+n^2$



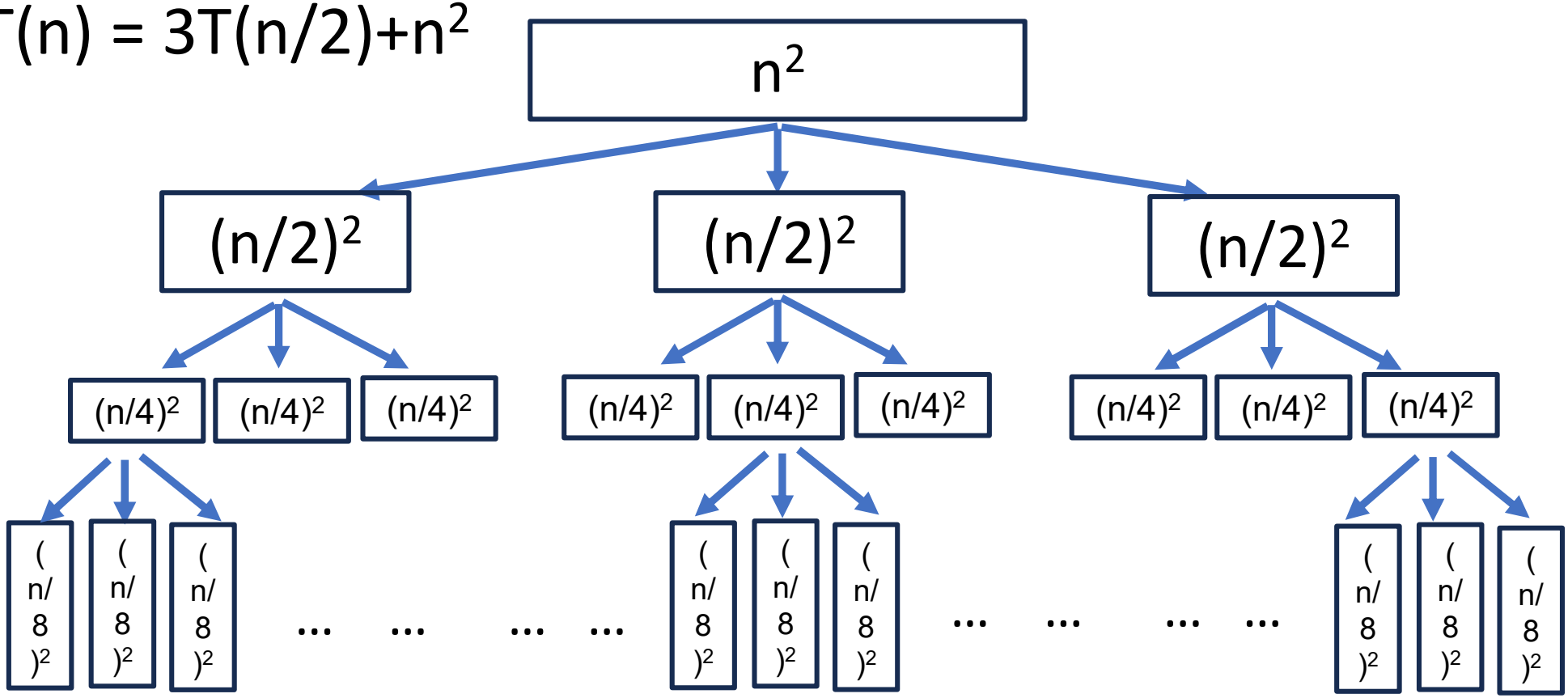
- 第i层的计算量: $\left(\frac{n}{2^i}\right)^2 \times 3^i = \left(\frac{3}{4}\right)^i n^2$

$T(n) = 3T(n/2)+n^2$



- 第i层的计算量: $\left(\frac{n}{2^i}\right)^2 \times 3^i = \left(\frac{3}{4}\right)^i n^2$
- 最底层是第几层? $\log_2 n$

$T(n) = 3T(n/2)+n^2$



- 第i层的计算量: $\left(\frac{n}{2^i}\right)^2 \times 3^i = \left(\frac{3}{4}\right)^i n^2$
- 最底层是第几层? $\log_2 n$
- 总计算量: $\sum_{i=0}^{\log_2 n} \left(\frac{3}{4}\right)^i n^2 =$?

$$\sum_{i=0}^{\log_2 n} \left(\frac{3}{4}\right)^i n^2 = \quad ?$$

求解等比数列的和:

$$\sum_{i=0}^{k-1} \left(\frac{3}{4}\right)^i = \frac{1-\left(\frac{3}{4}\right)^k}{1-\frac{3}{4}} = \frac{4}{1}\left(1-\left(\frac{3}{4}\right)^k\right)$$

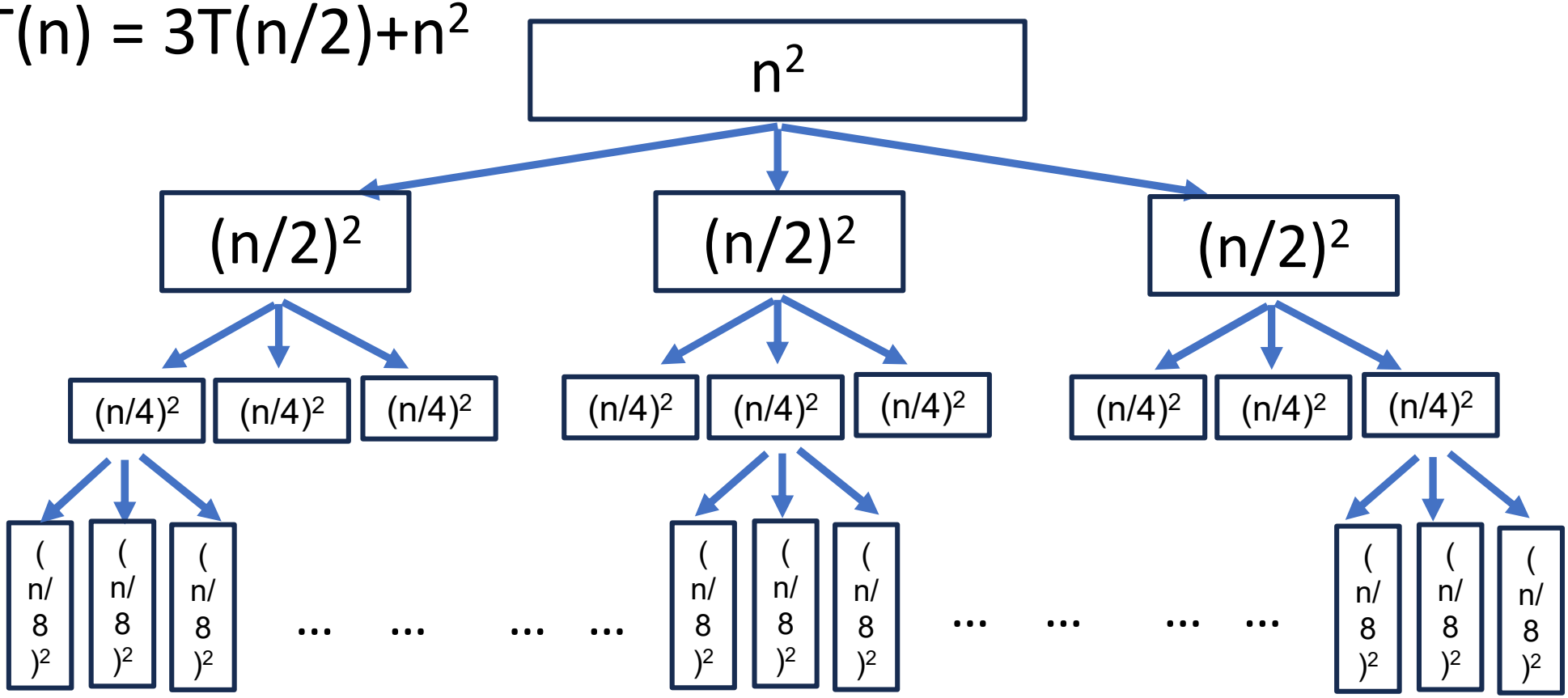
当 $k = \log_2 n$ 时，$\left(\frac{3}{4}\right)^k$ 趋近于 0，因此该求和项是一个常数:

$$\sum_{i=0}^{k-1} \left(\frac{3}{4}\right)^i = O(1)$$

因此:

$$\sum_{i=0}^{\log_2 n} \left(\frac{3}{4}\right)^i n^2 = O(n^2)$$

$T(n) = 3T(n/2)+n^2$



- 第i层的计算量: $\left(\frac{n}{2^i}\right)^2 \times 3^i = \left(\frac{3}{4}\right)^i n^2$
- 最底层是第几层? $\log_2 n$
- 总计算量: $\sum_{i=0}^{\log_2 n} \left(\frac{3}{4}\right)^i n^2 = O(n^2)$

假设归纳

$$T(n) = 2T(n-1)+1$$
$$T(1) = 0$$

YouTube频道: hwdong

假设归纳

- 先猜测其数量级的界（迭代展开或递归树），
- 再用数学归纳法证明。

$T(n) = 2T(n/2)+n-1$

$T(1)=0$

- 猜测其紧界为$\Theta(n \log n)$.即 $T(n) = \Theta(n \log n)$.
- 即存在正数c_1,c_2和n_0使得对所有$n \geq n_0$有：

$$c_1 n\log n \leq T(n) \leq c_2 n\log n$$

证明:

1) n=1,显然成立 $T(1)=0 = \Theta(1 \log 1)$
2) 假设<n时，都成立，则:

$T(n) = 2T(n/2)+n-1 = 2\,\Theta(\,(n/2)\log(n/2)\,)+n-1$

$= \Theta(n\,(\log n-\log 2)+n-1) = \Theta(n \log n)$

证明2：先证 T(n) = O(nlogn)

即要证存在c和n_0，当$n>n_0$， $T(n) \le c*n\log n$.

1) $T(1)=0 \le c*(1 \log 1)=0$
2) 设对小于<n的所有k， $T(k) \le c*k\log k$ 都成立， 则

$$T(n) = 2T(n/2)+n-1 \le 2\, c\, n/2 \log(n/2)+n-1$$

$$= cn \log (n/2)+n-1 = cn\log n - cn\log 2+n-1$$

只要 $-cn\log 2+n-1 \le 0$即可

即 只要$c \ge (n-1)/(n\log 2)$

取$c \ge n/(n\log 2)= 1/\log 2$的一个数即可。 比如$c = 1/\log 2$

证明2：再证 $T(n) = \Omega(n\log n)$

证明：证法类似

既然$T(n) = O(n\log n)$且 $T(n) = \Omega(n\log n)$。

因此：$T(n) = \Theta(n\log n)$。

$$\mathrm{T}(n) \le f(n) + \sum_{i=1}^{k} T(n_i)$$

猜测： $\mathrm{T}(n) \le \mathrm{O}(g(n))$

只要证明，存在常数c和正整数n_0，使得当$n \ge n_0$时，$T(n) \le c(g(n))$ 成立。

数学归纳法：

1）当$n = n_0$时成立 $\mathrm{T}(n_0) \le \mathrm{c}\,(g(n_0))$

2）设对小于n的n′， $\mathrm{T}(n') \le \mathrm{c}\,(g(n'))$

要证 $\mathrm{T}(n) \le \mathrm{c}\,(g(n))$

$$T(n) \le T(n/5) + T(7n/10) + an$$

猜测: $T(n) = O(n)$

分析: $T(n) \le T(n/5) + T(7n/10) + an$

$$\le c(n/5) + c(7n/10) + an$$

$$\le 9cn/10 + an$$

$$\le cn$$

$9c/10 + a \le c$ $\qquad$ $10a \le c$

假如$T(1) = 1$.则$T(1) = 1 \le c \cdot 1 \Rightarrow c \ge 1$

取c=max{10a，1}

$$T(n) \le T(n/5) + T(7n/10) + an$$

- 证：取c=max{10a，1}，$n_0 = 1$
- 1) $T(1) = 1 \le cn = c \cdot 1 = c$是成立
- 2) 设对小于n的n’，T(n’)<=cn’

$$\begin{aligned} T(n) &\le T(n/5) + T(7n/10) + an \\ &\le c(n/5) + c(7n/10) + an \\ &\le 9cn/10 + an = (9c/10 + a)n \\ &\le (9c/10 + c/10)n = cn \end{aligned}$$

高阶方程的化简

$$T(n) = 2T(n-1)+1$$
$$T(1) = 1$$

YouTube频道: hwdong

快速排序算法

快速排序 [**70**, 74,60,76, 83,72,55,65,79]

划分: [65, 55,60] , 70, [83,72,76,74,79]

快速排序 [65, 55,60]

[55, 60,65]

快速排序 [83,72,76,74,79]

[72, 74,76,79,83]

Qsort([70, 74,60,76, 83,72,55,65,79])

- 划分: [65, 55,60] , 70, [83,72,76,74,79]
- Qsort([65, 55,60])
  - 划分: [60, 55], 65, []
  - Qsort([60,55])
    - 划分: [55], 60, []
    - Qsort([55])
    - Qsort([])
  - Qsort([])
- Qsort([83,72,76,74,79])

.

.

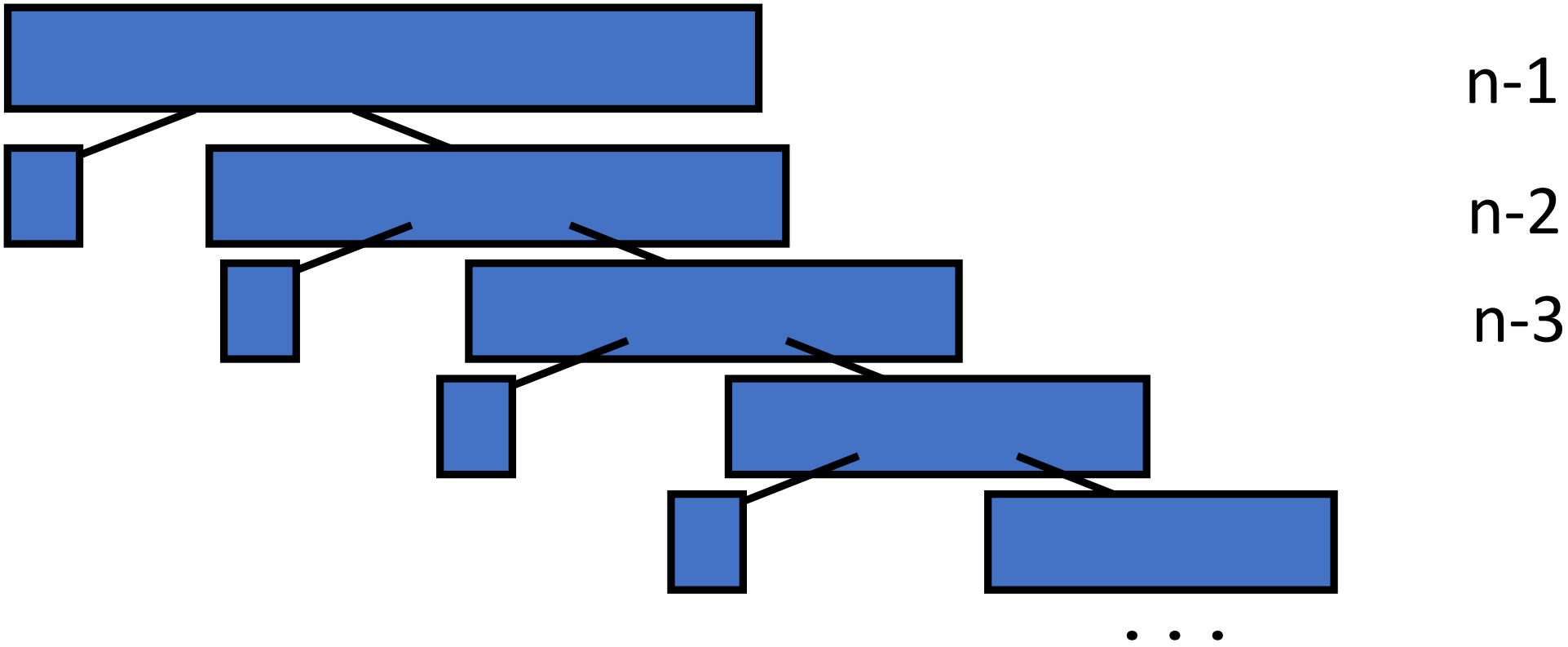
.

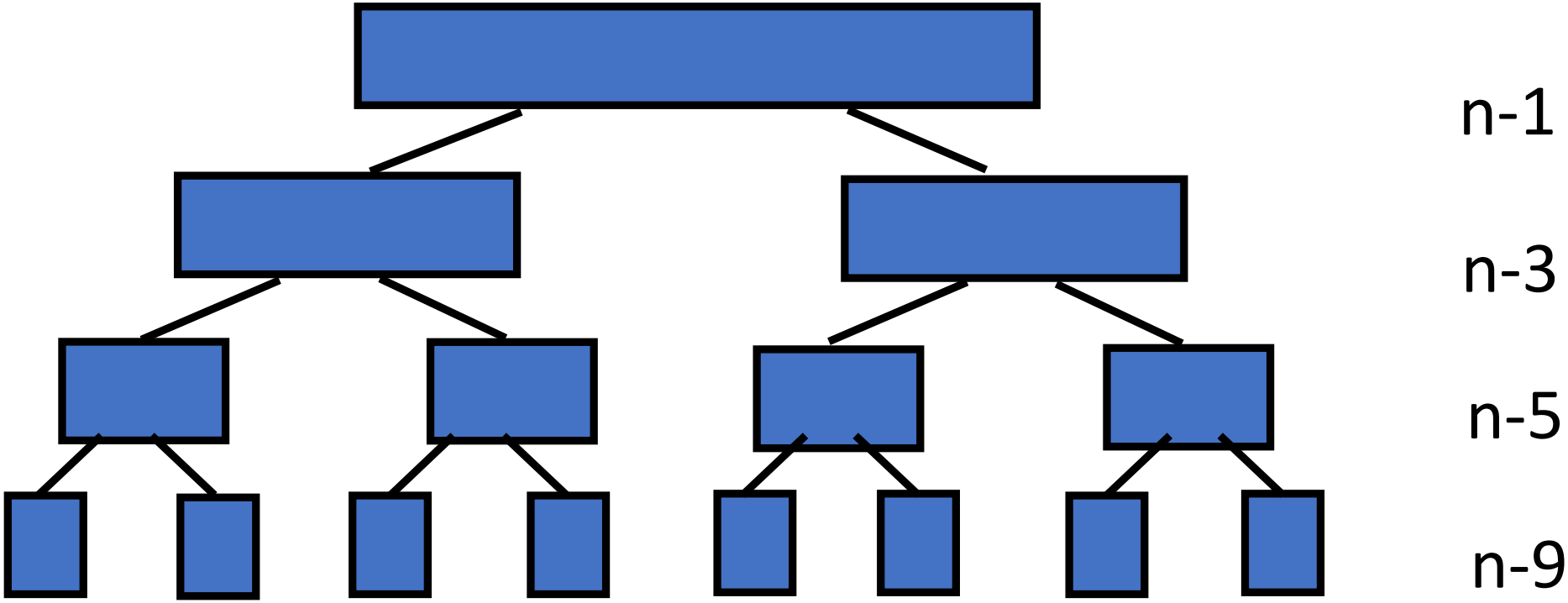
```
void QSort(T a[], int L, int H) {
   if(L < H){    //待排序数列长度大于1

      int pivotloc = Partition(a, L, H);

      //对左子序列进行快速排序
      QSort(a, L, pivotloc - 1);

         //对右子序列进行快速排序
      QSort(a, pivotloc + 1, H);
   }
}
```


n-1
n-2
n-3
. . .



$T(n) = 2T(\lfloor n/2 \rfloor)+n-1$

$T(n) = 2T(n/2)+n-1$

$T(n) = \Theta(n\log n)$

平均情况:

$T(0)+T(n-1)+n-1$

$T(1)+T(n-2)+n-1$

$T(2)+T(n-3)+n-1$

…

$T(n-1)+T(0)+n-1$

$$T(n) = 2/n\ (T(0)+T(1)+\ldots+T(n-1))+n-1$$

$$T(n) = 2/n\ (T(0)+T(1)+\ldots+T(n-1))+n-1$$

$$nT(n) = 2\ (T(0)+T(1)+\ldots+T(n-1))+ n(n-1)$$

$$(n-1)T(n-1) = 2\ (T(0)+T(1)+\ldots+T(n-2))+ (n-1)(n-2)$$

$$nT(n)-(n-1)T(n-1) = 2\ T(n-1)+ 2n-2$$

$$nT(n) = (n+1)T(n-1) + 2n-2$$

$$T(n)/(n+1) = T(n-1)/n + 2/(n+1)-2/(n(n+1))$$

$$T(n)/(n+1) = T(n-1)/n + 2/(n+1)-2(1/n-1/(n+1))$$

$$\mathbf{1/(n(n+1)) = 1/n-1/(n+1)}$$

$T(n)/(n+1) = T(n-1)/n + 2/(n+1)-2(1/n-1/(n+1))$

换元:

$W(n) = W(n-1)+ \ 2/(n+1))-2(1/n-1/(n+1))$

$= W(n-2) + \ 2/n+ \ 2/(n+1)-2(1/(n-1)-1/(n+1))$

$= W(1) + 2/3+\ldots+ \ 2/n+ \ 2/(n+1)-2(1- 1/(n+1))$

$= \Theta (\ln n) = \Theta (\log n) \quad => T(n) = \Theta (n\log n)$

$1/(n(n+1)) = 1/n-1/(n+1)$

$-2/(1*2)-2/2*3-\ldots-2/(n(n+1)) = -2(1-1/(n+1))$

$T(n)/(n+1) = T(n-1)/n + 2/(n+1)-2/(n(n+1))$

$= T(n-2)/(n-1) + 2/n-2/((n-1)n) + 2/(n+1)-2/(n(n+1))$

$= T(1)/2 + 2/3+2/4+\ldots2/(n+1)$

$-2/(1*2)-2/2*3-\ldots-2/(n(n+1))$

$= T(1)/2 + 2/3+2/4+\ldots2/(n+1) -2(1-1/(n+1))$

$= ½+ 2/3+2/4+\ldots2/(n+1) -2+2/(n+1)$

$= 2(1+1/2+1/3+1/(n+1))-4.5+2/(n+1)$

$= \Theta (\ln n) = \Theta (\log n)$

主定理

$T(n) = 2T(n-1)+1$

$T(1) = 1$

YouTube频道: hwdong

主定理　　多个同规模子问题

Let $T(n) = a \cdot T\left(\frac{n}{b}\right) + O(n^d)$ *be a recurrence where* $a \geq 1, b > 1$. *Then,*

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \\ O(n^d) & \text{if } a < b^d \\ O(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

也适用于子问题大小 $\lceil \frac{n}{b} \rceil$, $\lfloor \frac{n}{b} \rfloor$, *or* $\frac{n}{b} + 1$.

Let $T(n) = a \cdot T\left(\frac{n}{b}\right) + O(n^d)$ *be a recurrence where* $a \geq 1, b > 1$. *Then,*

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \\ O(n^d) & \text{if } a < b^d \\ O(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

- Mult1: $T(n) = 4T\left(\frac{n}{2}\right) + O(n)$.

The parameters are $a = 4, b = 2, d = 1$, so $a > b^d$,

hence $T(n) = O(n^{\log_2 4}) = O(n^2)$.

Let $T(n) = a \cdot T\left(\frac{n}{b}\right) + O(n^d)$ *be a recurrence where* $a \geq 1, b > 1$. *Then,*

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \\ O(n^d) & \text{if } a < b^d \\ O(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

- Karatsuba: $T(n) = 3T\left(\frac{n}{2}\right) + O(n)$.

  The parameters are $a = 3, b = 2, d = 1$, so $a > b^d$,

  hence $T(n) = O(n^{\log_2 3}) = O(n^{1.59})$.

Let $T(n) = a \cdot T\left(\frac{n}{b}\right) + O(n^d)$ *be a recurrence where* $a \geq 1, b > 1$. *Then,*

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \\ O(n^d) & \text{if } a < b^d \\ O(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

- MergeSort: $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$.

The parameters are $a = 2, b = 2, d = 1$, so $a = b^d$, hence $T(n) = O(n \log n)$.

Let $T(n) = a \cdot T\left(\frac{n}{b}\right) + O(n^d)$ *be a recurrence where* $a \geq 1, b > 1$. *Then,*

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \\ O(n^d) & \text{if } a < b^d \\ O(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

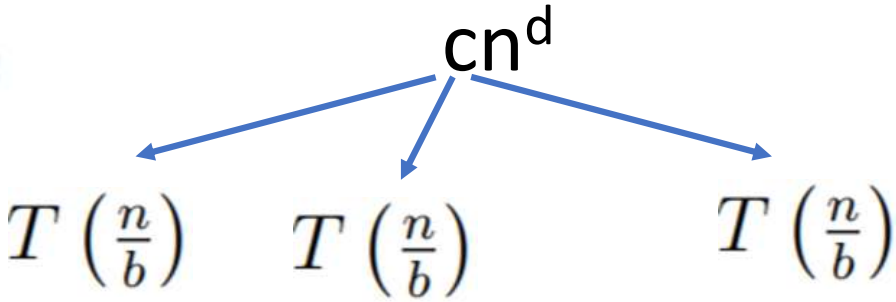
- Another example: $T(n) = 2T\left(\frac{n}{2}\right) + O(n^2)$.

The parameters are $a = 2, b = 2, d = 2$, so $a < b^d$,

hence $T(n) = O(n^2)$

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + O(n^d) \Rightarrow T(n) \leq a \cdot T\left(\frac{n}{b}\right) + c \cdot n^d$$

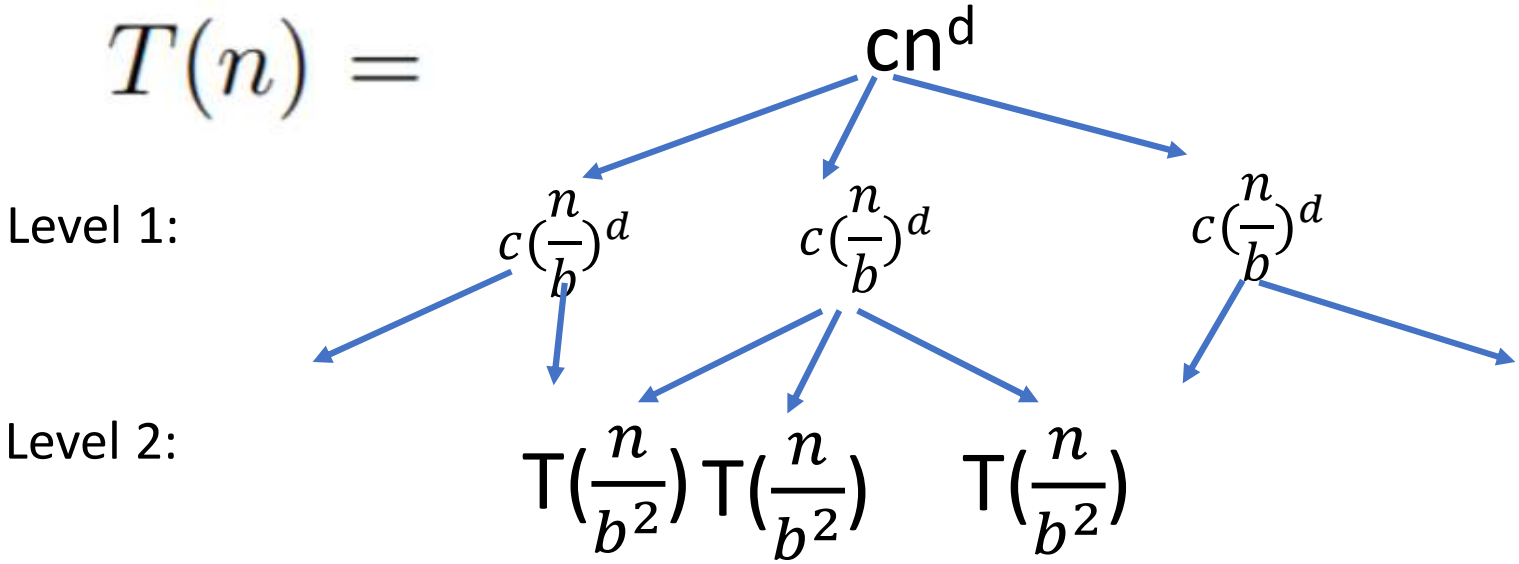
$$T(n) =$$



Level 1:

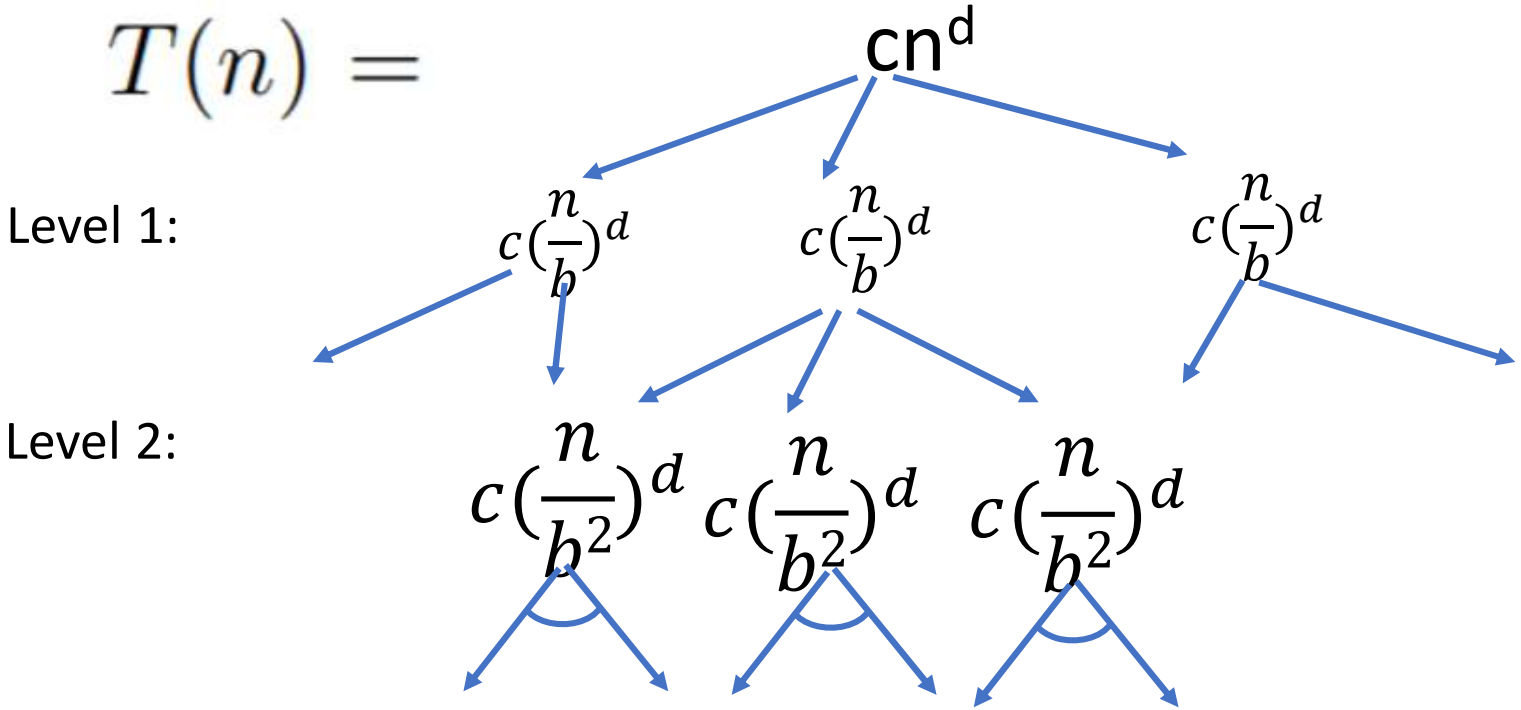
$$T(n) = a \cdot T\left(\frac{n}{b}\right) + O(n^d) \Rightarrow T(n) \leq a \cdot T\left(\frac{n}{b}\right) + c \cdot n^d$$

$$T(n) =$$



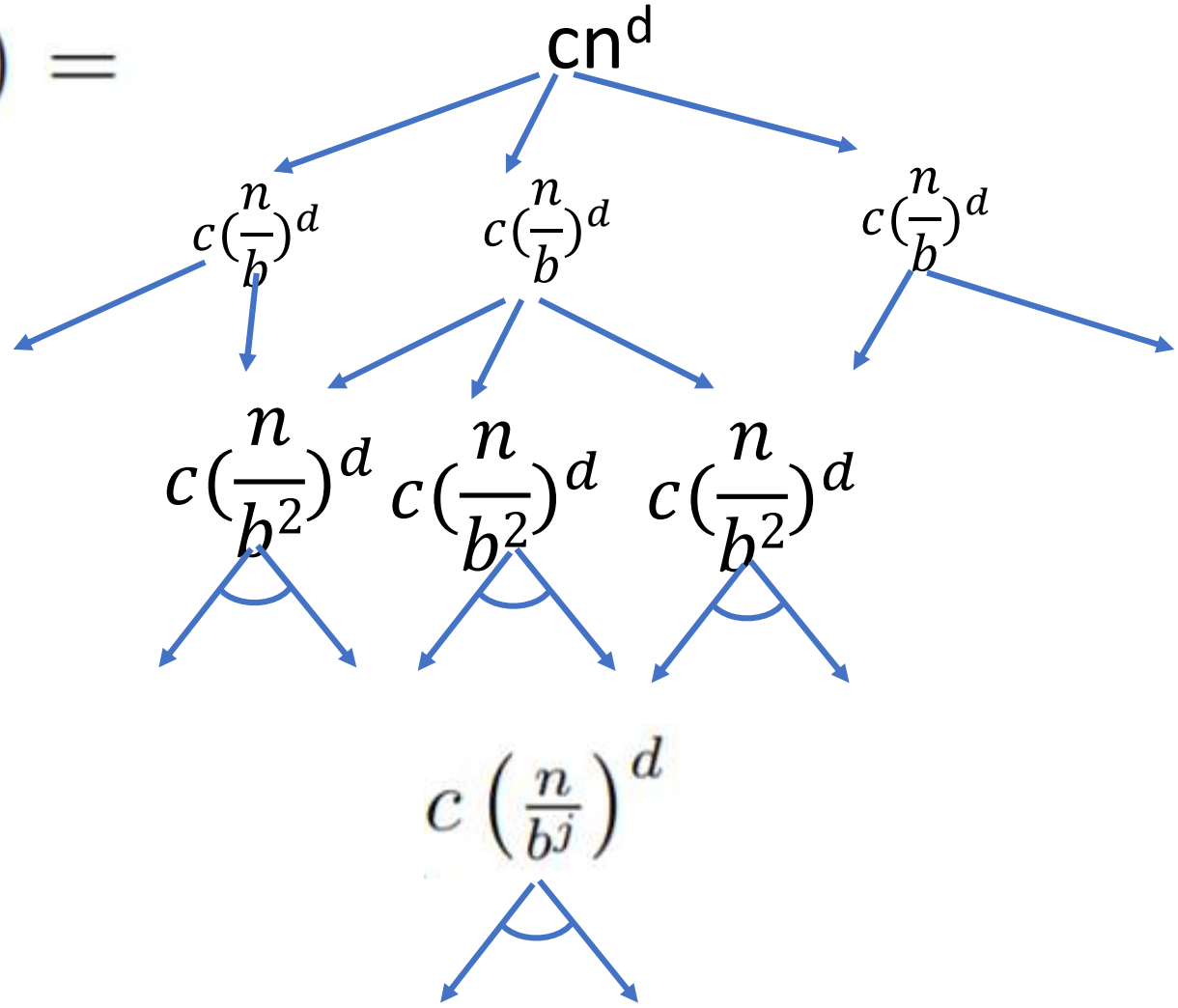
$$T(n) = a \cdot T\left(\frac{n}{b}\right) + O(n^d) \Rightarrow T(n) \leq a \cdot T\left(\frac{n}{b}\right) + c \cdot n^d$$

$T(n) =$



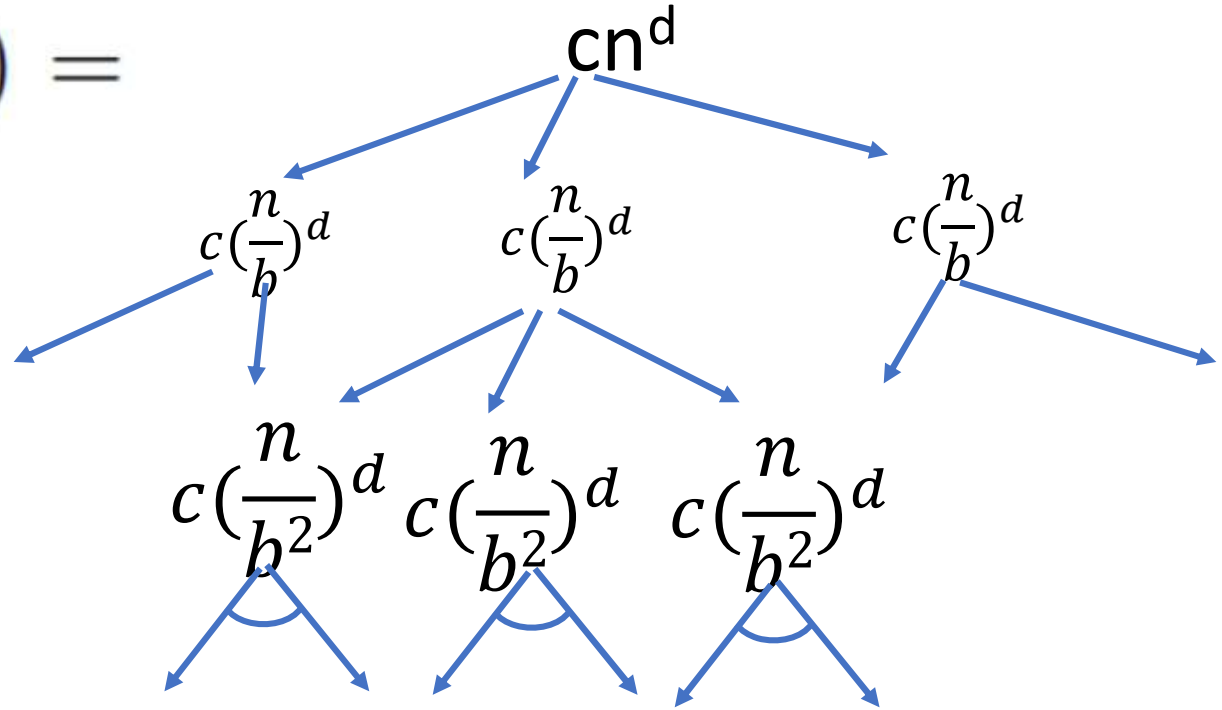
$$T(n) = a \cdot T\left(\frac{n}{b}\right) + O(n^d) \Rightarrow T(n) \leq a \cdot T\left(\frac{n}{b}\right) + c \cdot n^d$$

$T(n) =$



$$T(n) = a \cdot T\left(\frac{n}{b}\right) + O(n^d) \Rightarrow T(n) \leq a \cdot T\left(\frac{n}{b}\right) + c \cdot n^d$$

$T(n) =$



Level 1: $a.c(\frac{n}{b})^d$

Level 2: $a^2.c(\frac{n}{b^2})^d$

Level j: $c\left(\frac{n}{b^j}\right)^d$ $a^j \cdot c\left(\frac{n}{b^j}\right)^d$

每层 $a^j \cdot c\left(\frac{n}{b^j}\right)^d = cn^d\left(\frac{a}{b^d}\right)^j$

- 总的时间不超过: $cn^d \sum_{j=0}^{log_b n} \left(\frac{a}{b^d}\right)^j$

1. $a = b^d$. $(\log_b n + 1)cn^d = O(n^d \log n)$.

层数 每层

2. $a < b^d$.

$$\sum_{j=0}^{\log_b n} \left(\frac{a}{b^d}\right)^j \leq \sum_{j=0}^{\infty} \left(\frac{a}{b^d}\right)^j = \frac{1}{1-\frac{a}{b^d}} = \frac{b^d}{b^d - a}.$$

$$cn^d \cdot \frac{b^d}{b^d - a} = O(n^d).$$

3. $a > b^d$. In this case, $\sum_{j=0}^{\log_b n} \left(\frac{a}{b^d}\right)^j = \frac{\left(\frac{a}{b^d}\right)^{\log_b n+1} - 1}{\frac{a}{b^d} - 1}$.

the total work done is $O\left(n^d \cdot \left(\frac{a}{b^d}\right)^{\log_b n}\right) = O\left(n^d \cdot \frac{a^{\log_b n}}{b^{d \log_b n}}\right)$

$$= O\left(n^d \cdot \frac{n^{\log_b a}}{n^d}\right) = O(n^{\log_b a})$$

主定理2

Let $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$ *be a recurrence where* $a \geq 1$, $b > 1$. *Then,*

- *If* $f(n) = O\left(n^{\log_b(a)-\epsilon}\right)$ *for some constant* $\epsilon > 0$, $T(n) = \Theta\left(n^{\log_b(a)}\right)$.
- *If* $f(n) = \Theta\left(n^{\log_b(a)}\right)$, $T(n) = \Theta\left(n^{\log_b a} \log n\right)$.
- *If* $f(n) = \Omega\left(n^{\log_b(a)+\epsilon}\right)$ *for some constant* $\epsilon > 0$ *and if* $af(n/b) \leq cf(n)$ *for some* $c < 1$ *and all sufficiently large* n, *then* $T(n) = \Theta(f(n))$.

关注我

博客：hwdong.net

Youtube频道

hwdong

@hwdong · 5.01K subscribers · 558 videos

博客：https://hwdong-net.github.io >

youtube.com/c/4kRealSound and 4 more links